

Encoder-Level Lipschitz Regularization in STEM-GNN: Preliminary Results

Internal Research Memo
Working draft for discussion

September 7, 2026

Abstract

STEM-GNN’s finetune stage previously regularized only the linear classification head’s Lipschitz constant (via a Frobenius-norm penalty on its weight matrix, `decoder_jac_coeff`), while the GNN encoder beneath it remains unfrozen and unconstrained during finetuning. This memo reports a preliminary comparison of that head-only penalty against a new matching penalty applied to the encoder backbone’s own weight matrices (`encoder_lip_coeff`), added directly to `model/encoder.py`. Results are from a reduced-budget run on Cora and should be read as a first look, not a final claim.

1 Motivation

The finetune objective already included a Jacobian-style penalty on the decoder,

$$\mathcal{L}_{\text{head}} = \lambda_{\text{dec}} \|W_{\text{dec}}\|_F^2,$$

intended to bound the sensitivity of the final linear map to its input. However, during finetuning the encoder itself is not frozen (only the vector-quantizer is, by default) and carries no such constraint. An encoder that is free to amplify its input by an arbitrary factor can pass a perturbation through to the head no matter how tightly the head’s own weight norm is controlled, so a head-only penalty bounds only the last link in the chain. We added an analogous penalty over the encoder’s backbone weight matrices,

$$\mathcal{L}_{\text{enc}} = \lambda_{\text{enc}} \sum_{\ell} \|W_{\ell}\|_F^2,$$

summed over every weight matrix in the backbone’s message-passing layers (SAGE/GAT/GCN/GIN, including per-expert weights when MoE layers are used), excluding biases and GAT’s attention vectors. Both penalties are gated by independent coefficients and default to 0 (opt-in, no behavior change unless requested).

2 Experimental Setup

Honesty note on scope. This is a reduced-budget, single-dataset sanity check, not the project’s standard evaluation protocol. It was run on a single CPU core (no GPU available in this environment), which made the project’s default finetune budget (1000 epochs, early-stop patience 200, 10 splits per dataset, per config) impractical to run interactively. Numbers below should be treated as preliminary evidence about training dynamics, not final benchmark results.

- **Dataset:** Cora, node classification, all 10 splits shipped with the dataset (`finetune.py` always iterates over every provided split; there is no independent random seed here).
- **Backbone:** non-MoE GraphSAGE encoder (2 layers, hidden dim 768), loaded from the `default` pretrain checkpoint at epoch 50 (`ckpts/pretrain_model/default`).

- **Budget:** 60 finetune epochs, early-stop patience 30 – far short of the config-file default (1000 epochs / patience 200). Learning rate and all other hyperparameters use the `config/finetune.yaml` Cora defaults.
- **Coefficients:** a single untuned value, 10^{-3} , for `decoder_jac_coeff` and/or `encoder_lip_coeff` in each configuration below. No sweep over coefficient magnitude was performed.
- **Metric:** node-classification accuracy (%), mean $\pm$ standard deviation across the 10 splits, reported at each split’s best validation epoch (the project’s existing early-stopping selection rule).

3 Results

Configuration	λ_{dec}	λ_{enc}	Train	Val	Test
Baseline (no regularization)	0	0	98.43 $\pm$ 4.25	76.28 $\pm$ 3.34	75.87 $\pm$ 2.10
Head-only (prior behavior)	10^{-3}	0	98.43 $\pm$ 4.25	76.04 $\pm$ 3.35	75.71 $\pm$ 2.30
Encoder-only (new)	0	10^{-3}	99.64 $\pm$ 0.86	75.98 $\pm$ 2.82	75.78 $\pm$ 1.94
Head + Encoder (new, combined)	10^{-3}	10^{-3}	99.93 $\pm$ 0.21	75.96 $\pm$ 2.84	76.03 $\pm$ 0.88

4 Reading the Results

Clean-accuracy means are flat. All four configurations land within roughly one standard deviation of each other on test accuracy (75.7%–76.0%). At this coefficient and epoch budget, neither penalty moves mean clean accuracy on Cora in either direction. This is expected: Lipschitz regularization is a robustness/sensitivity control, not principally an accuracy lever, and clean i.i.d. accuracy is not the metric that should reveal its effect.

The head-only penalty changes nothing measurable. The head-only row is nearly identical to the baseline on every column, including the across-split standard deviation (train std 4.25 $\rightarrow$ 4.25, test std 2.10 $\rightarrow$ 2.30). This matches the original concern that motivated this change: constraining only the decoder’s weight norm has little effect when the unconstrained encoder is free to reshape the input beforehand.

The encoder-side penalty visibly changes training dynamics. Both configurations with $\lambda_{\text{enc}} > 0$ show a sharp drop in across-split variance on train accuracy (std 4.25 $\rightarrow$ 0.86 $\rightarrow$ 0.21) and a smaller but consistent drop in test-accuracy variance (std 2.10–2.30 $\rightarrow$ 1.94 $\rightarrow$ 0.88) as the encoder penalty is added and then combined with the head penalty. In other words, encoder-level regularization made the 10 finetuning runs converge more consistently with each other, even though it did not change their average outcome at this coefficient. That is a real, measured difference in training behavior attributable specifically to constraining the encoder, since the head-only run does not reproduce it.

This does not yet test the actual robustness claim. The motivation for encoder-level Lipschitz control is bounded sensitivity to input perturbations (missing features, dropped or shifted edges, degree/homophily shift) – not clean-split variance. STEM-GNN already ships scripts for exactly this (`scripts/missing_feature.py`, `scripts/random_edge_drop.py`, `scripts/degree_shift_ood.py`, `scripts/homophily_shift_ood.py`), none of which were run for this memo. The variance reduction observed above is suggestive but is not, by itself, a substitute for that evaluation.